

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ



HCM-UTE

**BÁO CÁO CUỐI KỲ MÔN HỌC
BLOCKCHAIN VÀ ỨNG DỤNG**

**HỢP ĐEN AN TOÀN VÀ ĐỊNH DANH
CHO ROBOT HỢP TÁC**

Nhóm thực hiện: Nhóm 9

Danh sách sinh viên:

1. Nguyễn Trần Minh Đức
2. Đỗ Thanh Sơn
3. Nguyễn Tuấn Kiệt
4. Huỳnh Trần Tiến Thịnh
5. Nguyễn Hữu Quý

MSSV 23139012

GVHD: TS. Huỳnh Thê Thiện

TP. Hồ Chí Minh, tháng 10 năm 2026

LỜI CẢM ƠN

Nhóm thực hiện xin cảm ơn thầy Huỳnh Thế Thiện đã hướng dẫn và cung cấp kiến thức trong học phần Blockchain và ứng dụng. Những góp ý của thầy giúp nhóm xây dựng kiến trúc hệ thống, triển khai các thành phần và hoàn thiện phương pháp đánh giá đề tài.

Trong quá trình thực hiện, nhóm còn hạn chế về điều kiện kiểm thử phần cứng và quy mô dữ liệu. Nhóm mong nhận được ý kiến đóng góp để tiếp tục hoàn thiện mô hình thực nghiệm và nội dung báo cáo.

TÓM TẮT

Đề tài SonarChain – HRC Safety Log xây dựng nguyên mẫu giám sát khoảng cách và lưu vết sự kiện trong bối cảnh cộng tác người–robot, kết hợp IoT, học máy tại biên và blockchain. Cảm biến HC-SR04 cung cấp dữ liệu khoảng cách cho ESP32. Tầng biên trích xuất đặc trưng, chạy mô hình phân loại và áp dụng ngưỡng nguy hiểm để phát cảnh báo tại chỗ. Gateway Python tiếp nhận telemetry, lưu SQLite và tạo gói bằng chứng ngoài chuỗi; các hợp đồng HRCSafetyLog và WorkPermitHandoff quản lý cam kết bằng chứng, phân quyền và quy trình cấp phép.

Thí nghiệm sử dụng 1.874 mẫu thuộc 17 bản ghi, được chia thành train, validation và test theo nguyên bản ghi. Trong bốn kiến trúc khảo sát, MLP 3–16–8–3 được chọn bằng Macro-F1 trên validation. Mô hình INT8 đạt accuracy 91,14% và Macro-F1 0,8937 trên 463 mẫu test; recall của EMERGENCY và APPROACHING lần lượt là 100% và 73,86%. Dự đoán float và INT8 trùng nhau ở 462/463 mẫu. Đây là kết quả offline của một phép chia dữ liệu; báo cáo chưa xác nhận accuracy, RAM hoặc thời gian suy luận của mô hình ứng viên trên ESP32 thực.

Hệ thống tách cảnh báo tại thiết bị khỏi giao dịch blockchain. Hàm băm hỗ trợ kiểm tra nội dung đã cam kết, còn trạng thái khóa trên chuỗi chỉ mang ý nghĩa logic. Nguyên mẫu sử dụng buzzer và chưa kiểm chứng mạch dừng motor/relay, do đó chưa được xem là hệ thống an toàn chức năng công nghiệp.

Từ khóa: ESP32; cảm biến siêu âm; TinyML; MLP; lượng tử hóa INT8; bằng chứng ngoài chuỗi; blockchain.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN	7
1.1. Bối cảnh và vấn đề đặt ra	7
1.2. Mục tiêu đề tài	7
1.3. Phạm vi và giới hạn	8
1.4. Bố cục báo cáo	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	9
2.1. Hệ thống thực-ảo và bối cảnh cộng tác người-robot	9
2.2. Đo khoảng cách bằng siêu âm và ESP32	9
2.3. MLP và phân loại ba trạng thái	10
2.4. Đặc trưng chuỗi thời gian và chia dữ liệu	10
2.5. TinyML và lượng tử hóa INT8	11
2.6. Các chỉ số đánh giá phân loại	11
2.7. Blockchain, hợp đồng thông minh và bằng chứng	11
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	13
3.1. Kiến trúc tổng thể	13
3.2. Tầng IoT và firmware	14
3.2.1. Cấu hình phần cứng	14
3.2.2. Chính sách trạng thái và xử lý lỗi	14
3.2.3. Gói telemetry	15
3.3. Pipeline AI	15
3.3.1. Dữ liệu và đặc trưng	15
3.3.2. Huấn luyện và chọn mô hình	15
3.3.3. Xuất INT8 và tích hợp firmware	16
3.4. Gateway và bằng chứng ngoài chuỗi	16
3.4.1. Tiếp nhận và lưu dữ liệu	16
3.4.2. Nội dung bằng chứng và outbox	16
3.4.3. Gửi giao dịch	17
3.5. Hai hợp đồng thông minh	17

3.5.1. HRCSafetyLog.....	17
3.5.2. WorkPermitHandoff	17
3.6. Dashboard	18
CHƯƠNG 4. KẾT QUẢ VÀ ĐÁNH GIÁ	19
4.1. Môi trường và chứng cứ thực nghiệm	19
4.2. Kết quả tăng IoT	20
4.3. Kết quả huấn luyện và lượng tử hóa	21
4.3.1. Phân bố dữ liệu	21
4.3.2. Diễn biến huấn luyện	21
4.3.3. Kết quả test float và INT8.....	22
4.3.4. Kết quả từng lớp và ma trận nhầm lẫn.....	22
4.3.5. Mô phỏng policy và giới hạn đánh giá	23
4.4. Kết quả kiểm thử phần mềm và hợp đồng	24
4.5. Kết quả dashboard và luồng tích hợp.....	25
4.6. Đối chiếu mục tiêu	26
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	27
5.1. Kết luận	27
5.2. Hạn chế	27
5.3. Hướng phát triển	27
TÀI LIỆU THAM KHẢO	29
PHỤ LỤC A. HƯỚNG DẪN TÁI LẬP	30
A.1. Chuẩn bị môi trường	30
A.2. Dashboard mô phỏng và blockchain local	30
A.3. Huấn luyện và kiểm thử từ thư mục gốc.....	30
A.4. Build và kiểm chứng candidate.....	30
PHỤ LỤC B. MANIFEST CHIA DỮ LIỆU	31
B.1. Train	31
B.2. Validation	31
B.3. Test	31

DANH MỤC TỪ VIẾT TẮT

Ký hiệu	Diễn giải
CPS	Cyber-Physical System — hệ thống thực-ảo
HRC	Human-Robot Collaboration — cộng tác người-robot
IoT	Internet of Things — Internet vạn vật
MLP	Multilayer Perceptron — mạng perceptron nhiều lớp
TinyML	Học máy với mô hình nhỏ, hướng tới suy luận trên vi điều khiển
INT8	Biểu diễn số nguyên có dấu 8 bit
EVM	Ethereum Virtual Machine — máy ảo Ethereum
RPC	Remote Procedure Call — gọi thủ tục từ xa
MQTT	Giao thức truyền thông theo cơ chế publish/subscribe
E-Stop	Emergency stop — dừng khẩn cấp; trong nguyên mẫu là trạng thái/yêu cầu dừng
SHA-256	Hàm băm mật mã cho giá trị băm 256 bit

CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN

1.1. Bối cảnh và vấn đề đặt ra

Trong môi trường sản xuất có sự cộng tác giữa người và robot, việc chia sẻ không gian làm việc đặt ra nhu cầu giám sát trạng thái vận hành và lưu lại dữ liệu khi xuất hiện tình huống nguy hiểm. Yêu cầu an toàn phải được đánh giá ở cấp ứng dụng và hệ thống robot, thay vì chỉ dựa vào một cảm biến hoặc một ngưỡng khoảng cách [2]. Trong phạm vi học phần, đề tài lựa chọn mô hình thu nhỏ để khảo sát sự kết hợp giữa đo khoảng cách, phân loại trạng thái và truy vết sự kiện.

Hai vấn đề được xem xét là phản ứng tại chỗ và khả năng đối chiếu dữ liệu sau sự kiện. Một tín hiệu cảnh báo có thể được phát nhanh tại thiết bị nhưng không cung cấp đủ thông tin để kiểm tra lịch sử xử lý. Ngược lại, nếu cảnh báo phải chờ dịch vụ mạng hoặc giao dịch blockchain, quá trình phản ứng sẽ phụ thuộc vào kết nối và thời gian xác nhận giao dịch. Vì vậy, hệ thống cần tách luồng cảnh báo cục bộ khỏi luồng lưu trữ và kiểm toán.

Nhật ký ngoài chuỗi giúp lưu dữ liệu chi tiết với chi phí thấp, nhưng cần có cơ chế phát hiện thay đổi nội dung. Cam kết bằng mã băm trên blockchain cho phép đối chiếu tệp bằng chứng với nội dung đã được ghi nhận. Cơ chế này chỉ hỗ trợ kiểm tra tính toàn vẹn; chất lượng phép đo và độ đúng của quyết định vẫn phụ thuộc vào cảm biến, dữ liệu, phần mềm và quy trình kiểm thử.

1.2. Mục tiêu đề tài

Mục tiêu tổng quát là xây dựng nguyên mẫu SonarChain với luồng dữ liệu từ cảm biến đến giao diện giám sát, đồng thời tách cảnh báo tại thiết bị khỏi hoạt động ghi nhận trên blockchain. Các mục tiêu cụ thể được ký hiệu R1–R7 để đối chiếu với kết quả trong Chương 4.

R1 — Thu thập khoảng cách bằng ESP32 và HC-SR04, phát telemetry và cảnh báo bằng buzzer; nhận biết trường hợp dữ liệu đo không hợp lệ.

R2 — Xây dựng pipeline phân loại SAFE, APPROACHING và EMERGENCY; đánh giá trên các bản ghi tách biệt và xuất mô hình INT8 phù hợp với TensorFlow Lite Micro.

R3 — Tiếp nhận telemetry qua gateway Python, bảo toàn trạng thái lỗi, lưu SQLite và cung cấp dữ liệu cho API/giao diện.

R4 — Chuẩn hóa nội dung bằng chứng, tạo SHA-256 và lưu gói JSON ngoài chuỗi với trạng thái xử lý có thể đối chiếu.

R5 — Xây dựng HRCSafetyLog để quản lý reporter, ghi digest và metadata, ngăn cam kết trùng và duy trì trạng thái khóa logic.

R6 — Xây dựng WorkPermitHandoff cho quy trình tạo, phê duyệt, vào khu vực, xác nhận bàn giao và hoàn tất giấy phép.

R7 — Xây dựng dashboard phân biệt nguồn mô phỏng và gateway, hiển thị trạng thái dữ liệu/bằng chứng và cung cấp hướng dẫn tái lập trên mạng EVM cục bộ.

1.3. Phạm vi và giới hạn

Phạm vi phần cứng gồm một ESP32, một HC-SR04, mạch chuyển mức, buzzer và nút điều khiển âm báo. Firmware chính sử dụng ESP-IDF và TensorFlow Lite Micro; đường truyền được đối chiếu trong báo cáo là USB Serial. Gateway có các thành phần hỗ trợ MQTT và mô hình phát hiện bất thường của phiên bản trước, nhưng chúng không phải nguồn của kết quả MLP INT8 được trình bày ở Chương 4.

Dữ liệu AI là các phiên đo khoảng cách được gán nhãn theo bản ghi. Mạng blockchain sử dụng Hardhat local, hai hợp đồng Solidity và MetaMask cho thao tác ký từ dashboard. Định danh trong nguyên mẫu là mã thiết bị, mã công việc/khu vực và tài khoản thực hiện giao dịch; chưa có cơ chế xác thực phần cứng hoặc nhận dạng người từ tín hiệu siêu âm.

Các ngưỡng 30 cm và 60 cm là tham số thử nghiệm, không phải khoảng cách bảo vệ đã được xác định theo tiêu chuẩn an toàn robot. Một cảm biến khoảng cách không phân biệt được người và vật. Nguyên mẫu chưa có bằng chứng về mạch E-Stop motor/relay, thời gian dừng cơ học hoặc chứng nhận an toàn chức năng. Kết quả accuracy được đo offline; chưa được sử dụng để khẳng định độ tin cậy trong môi trường công nghiệp.

1.4. Bố cục báo cáo

Báo cáo gồm năm chương. Chương 1 trình bày bối cảnh, mục tiêu và phạm vi. Chương 2 nêu cơ sở lý thuyết về CPS, đo khoảng cách, MLP/TinyML, đánh giá mô hình và blockchain. Chương 3 mô tả kiến trúc và thiết kế các thành phần. Chương 4 trình bày kết quả huấn luyện, kiểm thử phần mềm và giới hạn chứng cứ. Chương 5 tổng hợp kết luận, hạn chế và hướng phát triển. Phụ lục cung cấp lệnh tái lập và danh sách bản ghi của phép chia dữ liệu.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Hệ thống thực-ảo và bối cảnh cộng tác người-robot

Hệ thống thực-ảo (CPS) kết hợp thành phần vật lý và thành phần tính toán để thực hiện chức năng chung. Cảm biến cung cấp dữ liệu từ quá trình vật lý, phần mềm xử lý dữ liệu và cơ cấu chấp hành hoặc bộ phận cảnh báo tạo tác động trở lại. Những yếu tố như thời gian, độ tin cậy và khả năng quản lý dữ liệu cần được xem xét trong thiết kế CPS [1].

Trong SonarChain, HC-SR04 đo khoảng cách; ESP32 xử lý tín hiệu và điều khiển buzzer; gateway lưu và chuyển tiếp dữ liệu; blockchain ghi cam kết bằng chứng và trạng thái quy trình. Cách phân chia này cho phép phân tích riêng yêu cầu phản ứng tại thiết bị và yêu cầu lưu trữ, đối chiếu sau sự kiện.

Cộng tác người-robot đặt ra các yêu cầu an toàn ở cấp ứng dụng. ISO 10218-2:2025 đề cập yêu cầu đối với ứng dụng robot công nghiệp và robot cell [2]. Báo cáo chỉ sử dụng bối cảnh này để xác định vấn đề giám sát; chưa tuyên bố tuân thủ tiêu chuẩn. Muốn xác định khoảng cách bảo vệ thực tế còn cần biết tốc độ chuyển động, thời gian phản ứng/dừng, sai số đo và cấu hình robot, là các thông số chưa được đo trong nguyên mẫu.

2.2. Đo khoảng cách bằng siêu âm và ESP32

Cảm biến siêu âm ước lượng khoảng cách từ thời gian truyền âm đến vật phản xạ và quay lại. Với thời gian khứ hồi t và vận tốc âm c , khoảng cách được tính theo $d = c \cdot t / 2$. Ở khoảng 20 °C, c xấp xỉ 343 m/s; nếu t tính bằng microsecond thì d tính bằng centimet xấp xỉ $0,01715 \cdot t$. Nhiệt độ môi trường, góc bề mặt và khả năng phản xạ của vật ảnh hưởng đến phép đo [3].

HC-SR04 thông dụng có chân TRIG để kích hoạt và ECHO biểu diễn thời gian khứ hồi [13]. Firmware hiện sử dụng hệ số 0,0343/2 và giới hạn chờ ECHO. Nếu không có xung hợp lệ hoặc khoảng cách nằm ngoài miền kiểm tra, mẫu được đánh dấu lỗi thay vì tiếp tục dùng số đo cũ như dữ liệu mới.

ESP32 đảm nhiệm đọc GPIO, duy trì lịch sử đo, trích xuất đặc trưng và suy luận mô hình. Khi module HC-SR04 cấp nguồn 5 V tạo ECHO ở mức 5 V, tín hiệu phải đi qua mạch chuyển mức hoặc chia áp trước GPIO ESP32; mạch cần nối chung GND. Giới hạn điện áp GPIO được xác định theo datasheet ESP32 [4]. Việc tính đúng khoảng cách trong chương trình không thay thế cho kiểm tra điện áp và sai số đo trên mạch thực.

2.3. MLP và phân loại ba trạng thái

MLP là mạng nơ-ron gồm các lớp kết nối đầy đủ. Một lớp thực hiện phép biến đổi tuyến tính kết hợp hàm kích hoạt: $h = f(Wx + b)$, trong đó W là ma trận trọng số và b là vector bias. ReLU được định nghĩa bởi $f(z) = \max(0, z)$. Lớp cuối dùng softmax: $p_k = \exp(z_k) / \sum_j \exp(z_j)$, tạo các giá trị có tổng bằng 1 [5]. Các giá trị này là đầu ra của mô hình, chưa mặc nhiên là xác suất đã được hiệu chuẩn.

Bài toán của đề tài là học có giám sát với ba nhãn SAFE, APPROACHING và EMERGENCY. Cross-entropy của mẫu có nhãn y là $-\log(p_y)$. Kiến trúc được chọn có ba đầu vào, hai lớp ẩn 16 và 8 neuron, ba đầu ra; số tham số là $(3 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 3 + 3) = 227$. Mô hình được huấn luyện bằng Adam, kết hợp L2 và trọng số lớp để giảm ảnh hưởng của mất cân bằng dữ liệu [5].

Isolation Forest là thuật toán phát hiện bất thường bằng cách cô lập mẫu qua các phép chia ngẫu nhiên; mẫu khác biệt thường có đường đi ngắn hơn [14]. Trong project, đây là phương án hỗ trợ của gateway ở các phiên bản trước. Phát hiện bất thường và phân loại có giám sát là hai bài toán khác nhau, nên kết quả MLP không được dùng để đánh giá Isolation Forest.

2.4. Đặc trưng chuỗi thời gian và chia dữ liệu

Ba đặc trưng được tạo từ các mẫu hiện tại và quá khứ của cùng một bản ghi. Đặc trưng `distance_cm` là khoảng cách d_t . Đặc trưng `distance_delta_3` là $d_t - d_{t-3}$, được đặt bằng 0 khi chưa đủ ba mẫu quá khứ. Giá trị này có đơn vị cm và chưa chia thời gian, nên không phải vận tốc cm/s.

Đặc trưng `distance_std_5` là độ lệch chuẩn mẫu của tối đa năm khoảng cách gần nhất: $s = \sqrt{[\sum_i (d_i - \bar{d})^2 / (n-1)]}$, với n từ 2 đến 5; khi $n = 1$, pipeline đặt $s = 0$. Cách tính tương ứng rolling window với `ddof = 1`. Các đặc trưng được chuẩn hóa theo $x' = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$; trung bình và độ lệch chuẩn chỉ được tính từ train. Quy tắc này phải giống giữa Python và firmware.

Các mẫu liên tiếp của cùng một phiên đo thường có tương quan. Nếu chia ngẫu nhiên từng hàng, train và test có thể chứa thông tin gần trùng nhau. Vì vậy, đề tài chia theo `source_file` để mỗi bản ghi chỉ thuộc một tập; sau đó mới tạo đặc trưng trong từng bản ghi. Cách đánh giá trên dữ liệu có nhóm được trình bày trong tài liệu scikit-learn [8]. Validation dùng chọn kiến trúc và theo dõi huấn luyện; test dùng đánh giá sau khi chốt lựa chọn.

Cửa sổ theo số mẫu phụ thuộc vào nhịp lấy mẫu. Thay đổi chu kỳ thu nhận làm thay đổi khoảng thời gian mà delta và độ lệch chuẩn đại diện. Do đó, trước khi triển khai cần đổi cadence của dữ liệu huấn luyện với cadence đo thực tế trên ESP32.

2.5. TinyML và lượng tử hóa INT8

TinyML hướng tới thực hiện suy luận bằng mô hình nhỏ trên thiết bị có tài nguyên hạn chế. Trong project, mô hình được huấn luyện ngoài thiết bị, chuyển sang TFLite và dự kiến chạy bằng TensorFlow Lite Micro trên ESP32. Bước lượng tử hóa sau huấn luyện dùng dữ liệu hiệu chuẩn lấy từ train để ước lượng dải giá trị tensor; converter giới hạn phép toán INT8 và đặt cả input/output là int8 [6].

Quan hệ lượng tử hóa được biểu diễn gần đúng bởi $x \approx (q - z) \cdot s$, trong đó q là số nguyên, s là scale và z là zero-point [7]. Với đầu vào đã chuẩn hóa x' , firmware tính $q = \text{clip}(\text{round}(x'/s) + z, -128, 127)$. Scale và zero-point được đọc từ tensor metadata. Đầu ra được giải lượng tử hóa khi cần kiểm tra giá trị; lớp dự đoán được chọn theo argmax.

INT8 có thể giảm dung lượng trọng số và phù hợp với phép toán trên vi điều khiển, nhưng phải đánh giá lại bằng TFLite interpreter để kiểm tra thay đổi dự đoán. Kích thước tệp model không bằng RAM sử dụng của hệ thống; RAM còn phục vụ activation, tensor arena, tác vụ và các thành phần khác. Vì vậy, phép đo RAM và thời gian suy luận trên board là bước kiểm chứng riêng.

2.6. Các chỉ số đánh giá phân loại

Ma trận nhầm lẫn C được quy ước có hàng là lớp thật và cột là lớp dự đoán. Với N mẫu, $\text{accuracy} = \sum_k C_{kk}/N$. Với mỗi lớp k , $\text{precision}_k = TP_k/(TP_k + FP_k)$, $\text{recall}_k = TP_k/(TP_k + FN_k)$, và $F1_k = 2TP_k/(2TP_k + FP_k + FN_k)$. Macro-F1 là trung bình F1 của ba lớp, không phải F1 tính từ precision và recall trung bình [9].

Accuracy phản ánh tỷ lệ dự đoán đúng toàn bộ mẫu, còn Macro-F1 cho mỗi lớp vai trò như nhau. Recall APPROACHING giúp nhận diện khả năng bỏ sót cảnh báo sớm; recall EMERGENCY đánh giá bỏ sót lớp nguy hiểm trong tập kiểm thử; precision EMERGENCY phản ánh mức báo động nhầm. Các chỉ số phải được báo cùng số mẫu và số phiên đo, tránh diễn giải nhiều mẫu liên tiếp như nhiều tình huống độc lập.

2.7. Blockchain, hợp đồng thông minh và bằng chứng

Blockchain tổ chức sổ cái theo các khối liên kết bằng mã băm, với việc cập nhật trạng thái được xử lý theo quy tắc của mạng. Hợp đồng thông minh là chương trình quản lý trạng thái và kiểm tra điều kiện khi giao dịch được thực thi trên EVM. Hợp đồng không tự đọc cảm biến ngoài chuỗi; gateway hoặc tài khoản được cấp quyền phải gửi

dữ liệu vào [10]. Việc tiêu thụ tài nguyên được đo bằng gas; gas sử dụng và giá gas là hai yếu tố khác nhau.

Trong SonarChain, HRCSafetyLog chỉ chấp nhận owner hoặc reporter hợp lệ cho việc ghi bằng chứng. WorkPermitHandoff kiểm tra các vai trò supervisor, gateway và người tham gia tương ứng với từng thao tác. Địa chỉ tài khoản thể hiện bên gửi giao dịch; mã băm device_id chỉ là định danh quy ước, chưa xác thực phần cứng tạo dữ liệu. Các trạng thái deviceLocked và emergencyStopByDevice là khóa logic, không chứng minh cơ cấu vật lý đã dừng.

SHA-256 là hàm băm mật mã tạo digest 256 bit, tương đương 32 byte [11]. Hàm băm không phải mã hóa hoặc chữ ký số. Dự án chuẩn hóa JSON theo quy tắc xác định rồi băm phần nội dung bằng chứng; evidence_hash, evidence_status và tx_hash không tham gia phép băm. Vì vậy, trạng thái gửi có thể được cập nhật mà không đổi cam kết nội dung. Đây là quy tắc biểu diễn của project, chưa được tuyên bố là triển khai đầy đủ RFC 8785.

Khi kiểm toán, hệ thống tạo lại digest từ nội dung ngoài chuỗi và so sánh với digest đã ghi. Kết quả trùng khớp hỗ trợ kiểm tra nội dung so với cam kết, nhưng không chứng minh cảm biến đo đúng, nhãn đúng hoặc robot đã dừng. Bằng chứng chỉ hữu ích khi tệp ngoài chuỗi vẫn được lưu và cam kết trên chain có thể truy cập.

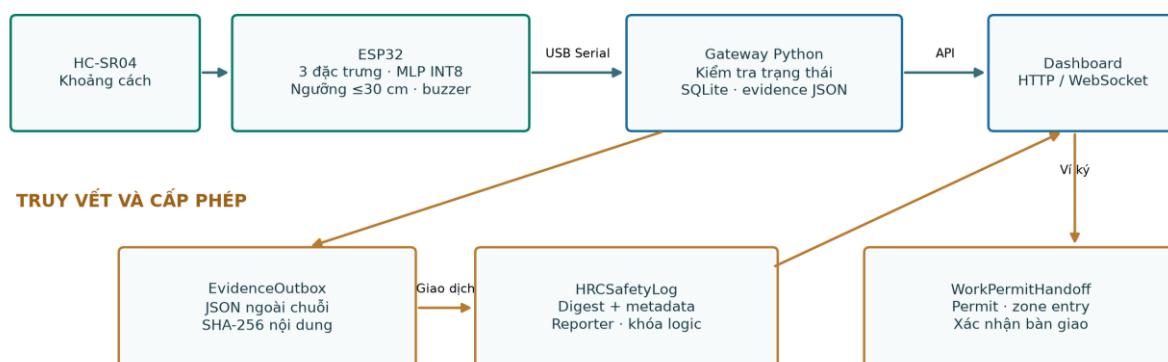
Mạng Hardhat cục bộ phục vụ phát triển và có thể reset trạng thái [12]. Vì thế, thí nghiệm trên mạng này chứng minh việc kiểm tra giao dịch và truy vết trong phiên chạy, chưa chứng minh tính bất biến tuyệt đối, độ bền hoặc mức phân tán của một mạng triển khai thực.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể

SonarChain tách luồng cảnh báo tại thiết bị khỏi luồng truy vết. ESP32 đọc khoảng cách, tạo đặc trưng, chạy model và phát buzzer theo trạng thái. Gateway tiếp nhận JSON qua Serial, kiểm tra dữ liệu, tạo bằng chứng và lưu SQLite; API và WebSocket cung cấp dữ liệu cho dashboard. Giao dịch blockchain chỉ được thực hiện khi có cấu hình và quyền tương ứng, nên không phải điều kiện để phát cảnh báo cục bộ.

CẢNH BÁO TẠI THIẾT BỊ



Blockchain phục vụ truy vết; trạng thái trên chain chưa chứng minh robot đã dừng vật lý.

Hình 3.1. Kiến trúc SonarChain sau khi đồng bộ pipeline TinyML và luồng truy vết.

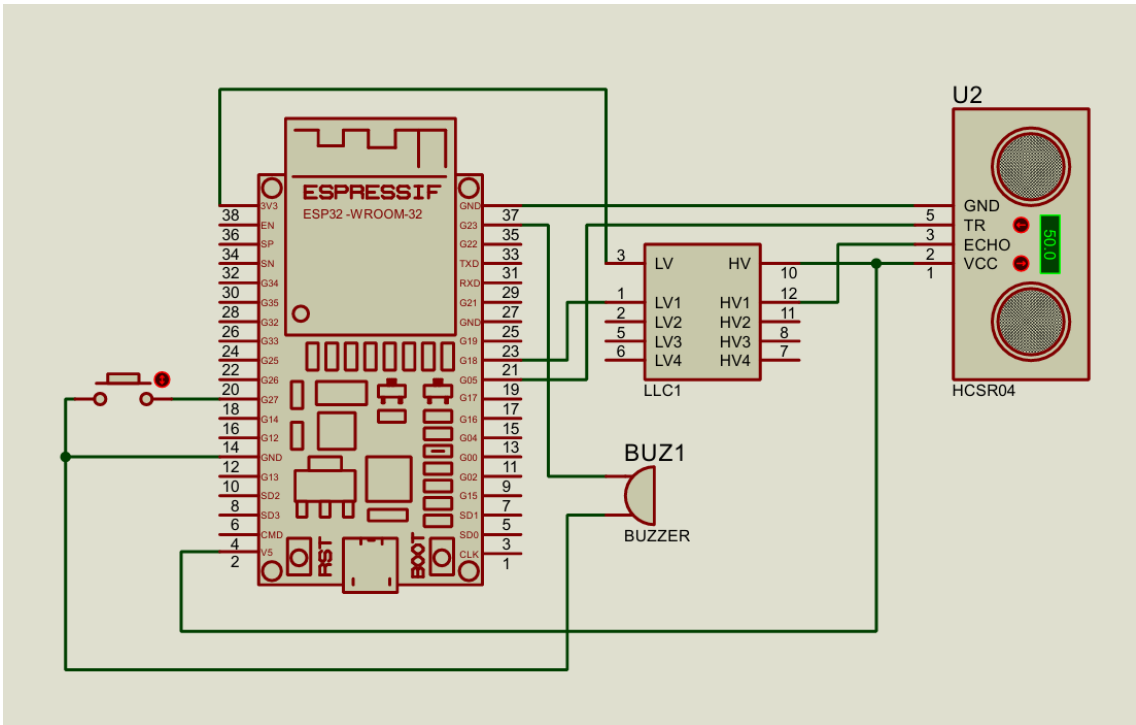
Bảng 3.1. Chức năng các luồng

Luồng	Xử lý	Kết quả
Cảnh báo cục bộ	Đọc cảm biến, xử lý lỗi, trích đặc trưng, MLP và ngưỡng ≤ 30 cm	Trạng thái và buzzer; cờ yêu cầu dừng
Dữ liệu/kiểm toán	Serial $\rightarrow$ gateway $\rightarrow$ SQLite/evidence $\rightarrow$ API; gửi chain khi cấu hình	Lịch sử mẫu, JSON, digest và trạng thái giao dịch
Cấp phép	Tài khoản tạo, phê duyệt, vào vùng và xác nhận bàn giao	Trạng thái permit và sự kiện Solidity

Luồng cảnh báo hiện chưa có motor/relay. Mất kết nối blockchain không làm chương trình ESP32 chờ giao dịch, nhưng chưa đủ để kết luận hệ thống an toàn khi mất nguồn hoặc lỗi phần cứng. Khóa logic trên chain và trạng thái tức thời trên ESP32 được theo dõi riêng.

3.2. Tầng IoT và firmware

3.2.1. Cấu hình phần cứng



Hình 3.2. Sơ đồ phần cứng trong báo cáo gốc: ESP32, HC-SR04, chuyển mức, buzzer và nút.

Bảng 3.2. Pin map của firmware chính

Thành phần	GPIO/kết nối	Vai trò
HC-SR04 TRIG	GPIO5	Kích hoạt phép đo
HC-SR04 ECHO	GPIO18 qua chuyển mức	Đo thời gian xung; bảo vệ mức logic ESP32
Buzzer	GPIO23	Phát âm báo
Nút	GPIO27 → GND	Bật/tắt âm báo; không phải reset E-Stop vật lý
Nguồn/GND	Theo mạch thực tế, chung GND	Cấp nguồn module và tham chiếu tín hiệu

3.2.2. Chính sách trạng thái và xử lý lỗi

Firmware ưu tiên ngưỡng cứng: nếu khoảng cách hợp lệ ≤ 30 cm, trả EMERGENCY. Ngoài ngưỡng này, khi model hợp lệ, trạng thái lấy từ SAFE/APPROACHING/EMERGENCY của AI; dự đoán EMERGENCY không bị hạ xuống APPROACHING. Model chưa sẵn sàng hoặc suy luận lỗi trả AI_FAULT; mẫu không hợp lệ trả SENSOR_FAULT và reset lịch sử đặc trưng.

Buzzer bật khi EMERGENCY, SENSOR_FAULT hoặc AI_FAULT, trừ khi người dùng đang tắt âm báo. Khi có mẫu SAFE, trạng thái tắt âm được xóa. Cờ

emergency_stop của firmware được đặt khi trạng thái là EMERGENCY; các mẫu lỗi không được trình bày như SAFE. Firmware có thời gian chờ 100 ms sau mỗi lượt xử lý, nên chu kỳ thực tế còn gồm thời gian đọc và suy luận.

3.2.3. Gói telemetry

Dữ liệu phát ra dưới dạng một JSON mỗi dòng, gồm device_id, sensor_id, timestamp_ms, distance_cm, state, emergency_stop, buzzer_on, buzzer_silenced và seq. timestamp_ms là uptime của ESP32, không phải Unix timestamp; gateway cần bảo toàn hoặc bổ sung mốc thời gian nhận theo quy tắc xử lý. Firmware hiện chưa phát vector xác suất của MLP hoặc hash phiên bản model trong telemetry.

```
{ "device_id": "ESP32-HRC-01", "sensor_id": "HC-SR04",  
  "timestamp_ms": 1234, "distance_cm": 24.7, "state": "EMERGENCY",  
  "emergency_stop": true, "buzzer_on": true,  
  "buzzer_silenced": false, "seq": 12 }
```

3.3. Pipeline AI

3.3.1. Dữ liệu và đặc trưng

tinyml_pipeline.py kiểm tra các cột source_file, elapsed_s, distance_cm và label; từ chối dữ liệu thiếu, không hữu hạn hoặc thời gian không tăng trong một bản ghi. Dataset chỉ chứa phép đo hợp lệ trong miền kiểm tra 2–450 cm và ba nhãn. SENSOR_FAULT/AI_FAULT được xử lý bằng logic hệ thống, không được gộp vào SAFE để huấn luyện.

Các bản ghi được chia theo lớp với seed 42, bảo đảm không trùng giữa ba tập và mỗi tập có đủ ba nhãn. Đặc trưng causal được tạo riêng trong từng bản ghi theo thứ tự distance_cm, distance_delta_3, distance_std_5. Trung bình và độ lệch chuẩn chuẩn hóa được fit trên train; validation/test chỉ được transform.

3.3.2. Huấn luyện và chọn mô hình

Bảng 3.3. Các kiến trúc khảo sát

Ứng viên	Kiến trúc	Số tham số
Linear softmax	$3 \rightarrow 3$	12
MLP 4	$3 \rightarrow 4 \rightarrow 3$	31
MLP 8	$3 \rightarrow 8 \rightarrow 3$	59
MLP 16–8	$3 \rightarrow 16 \rightarrow 8 \rightarrow 3$	227

train_keras_tflite.py huấn luyện các ứng viên với Adam learning rate 0,001, batch size 32, tối đa 150 epoch, L2 = 0,001 tại các lớp ẩn và class weight cân bằng từ train. EarlyStopping theo val_loss có patience 20, min_delta 0,0001 và restore_best_weights;

ReduceLRonPlateau có patience 7, factor 0,5 và learning rate tối thiểu 10^{-5} . Sau từng lần huấn luyện, model được đánh giá trên validation để chọn theo Macro-F1; tiêu chí phụ là recall EMERGENCY và số tham số nhỏ hơn.

Test được chạy sau khi chọn ứng viên. Notebook xuất lịch sử huấn luyện, manifest split, metrics, metadata và model. Các metric validation của model sau phục hồi trọng số cần phân biệt với log của epoch cuối; không dùng log cuối của những ứng viên dừng sớm làm bằng so sánh model đã chốt.

3.3.3. Xuất INT8 và tích hợp firmware

Representative dataset lấy tối đa 500 mẫu đã chuẩn hóa từ train. Converter đặt TFLITE_BUILTINS_INT8 cùng input/output int8. Pipeline chạy TFLite interpreter trên toàn bộ test, ghi dự đoán và so sánh với float. Model C++ và mean/scale được xuất cùng nhau; firmware kiểm tra tensor input/output [1,3], dtype và quantization metadata trước khi dùng.

Trong lần rà soát, ứng viên mới được lưu riêng tại ai_model/artifacts/esp32_candidate/. File model_data.cc đang có trong firmware không tự được thay bằng candidate. Việc dùng ứng viên đòi hỏi copy đồng thời model_data.cc và model_data.h, build lại, nạp board và chạy kiểm chứng parity/hiệu năng. Báo cáo không coi lần build firmware cũ là phép thử accuracy của model mới.

3.4. Gateway và bằng chứng ngoài chuỗi

3.4.1. Tiếp nhận và lưu dữ liệu

Serial reader bỏ qua dòng boot/debug và tiếp nhận JSON telemetry. Hàm process_row kiểm tra khoảng cách, áp dụng ngưỡng gateway 30/60 cm, bảo toàn EMERGENCY từ edge và ánh xạ APPROACHING thành WARNING khi cần. SENSOR_FAULT/AI_FAULT được giữ dưới dạng trạng thái lỗi trong pipeline lưu trữ/giao diện; không tái sử dụng khoảng cách cũ như mẫu mới.

TelemetryStore lưu SQLite với khóa event_id và các trường thiết bị, thời gian, khoảng cách, severity, emergency_stop, confidence, evidence_hash, evidence_status, tx_hash và raw_payload. API cung cấp latest/history và WebSocket. Một số confidence trong gateway là giá trị quy ước của policy, chưa phải xác suất đã hiệu chuẩn hoặc xác suất softmax được đọc trực tiếp từ ESP32.

3.4.2. Nội dung bằng chứng và outbox

build_evidence tạo envelope theo schema sonarchain.evidence.v1, gồm mã sự kiện/thiết bị/cảm biến, thời điểm đo/nhận, khoảng cách, trạng thái, cờ dừng, confidence,

policy_version, model_version và source. Nội dung được biểu diễn bằng JSON với sort_keys=True, ensure_ascii=True, separators=(",", ":") và UTF-8. evidence_digest bỏ evidence_hash, evidence_status và tx_hash trước khi tính SHA-256.

EvidenceOutbox ghi JSON theo digest, kiểm tra lại hash trước gửi và quản lý trạng thái queued, pending, confirmed hoặc offchain. Trạng thái gửi và tx_hash là metadata vận hành nằm ngoài digest. Việc kiểm toán cần đối chiếu digest với cam kết tin cậy bên ngoài tệp; tự thay nội dung rồi tự tạo lại hash trong tệp không chứng minh nội dung mới là bản đã cam kết.

3.4.3. Gửi giao dịch

SafetyLogClient nạp ABI và địa chỉ hợp đồng, tạo tài khoản từ khóa riêng trong cấu hình và gửi recordEvidence. SHA-256 64 ký tự hex được chuyển sang bytes32; device_id và schema dùng keccak256 để ánh xạ sang mã Solidity. measuredAt và recordedAt có ý nghĩa khác nhau: thời điểm đo do nguồn/gateway cung cấp, còn thời điểm ghi lấy từ block.timestamp. Receipt thành công xác nhận giao dịch được thực thi; không xác nhận độ đúng vật lý của mẫu đo.

3.5. Hai hợp đồng thông minh

3.5.1. HRCSafetyLog

HRCSafetyLog quản lý owner/reporter, eventExists, evidenceExists, deviceLocked và emergencyStopByDevice. recordEvidence kiểm tra schema, digest/device hash khác 0, thời điểm đo dương, không trùng cam kết và tính nhất quán giữa severity với emergency_stop. Khi còi dừng được ghi, khóa logic được bật và không tự gỡ bởi sự kiện SAFE tiếp theo; clearEmergencyStop chỉ dành cho owner.

3.5.2. WorkPermitHandoff

WorkPermitHandoff quản lý supervisor, gateway, permit và handoff. Trạng thái được triển khai theo vòng đời PENDING → APPROVED → ACTIVE → COMPLETED; supervisor có thể chuyển permit phù hợp sang REVOKED. Hợp đồng kiểm tra thời hạn ở bước recordZoneEntry, sau đó kiểm tra người tham gia ở bước handoff/complete.

ZoneEntryRecorded chứa telemetryEventHash, nhưng contract hiện chỉ yêu cầu hash khác 0, chưa kiểm tra hash đó đã tồn tại trong HRCSafetyLog. Các bước handoff/complete cũng chưa bổ sung kiểm tra hết hạn giống bước entry. Vì vậy, quy trình là xác nhận của các tài khoản được cấp quyền, chưa tự chứng minh người đã vào vùng hoặc robot đã bàn giao vật lý.

Bảng 3.4. Phân quyền thao tác

Hợp đồng/thao tác	Quyền	Ý nghĩa
HRCSafetyLog: setReporter, clearEmergencyStop	Owner	Quản trị reporter/gỡ khóa logic
HRCSafetyLog: recordEvidence	Owner hoặc reporter	Ghi cam kết bằng chứng
WorkPermitHandoff: approve/revoke	Owner hoặc supervisor	Phê duyệt/thu hồi permit
WorkPermitHandoff: recordZoneEntry	Owner hoặc gateway	Kích hoạt permit trong thời hạn
WorkPermitHandoff: confirmHandoff	Worker/requester/supervisor	Xác nhận bàn giao
WorkPermitHandoff: completePermit	Worker/requester/owner	Hoàn tất permit đang ACTIVE

3.6. Dashboard

Dashboard HTML/JavaScript có hai nguồn dữ liệu riêng: mô phỏng và ESP32 qua gateway. Khi chọn gateway, nút tạo mẫu giả bị khóa; dữ liệu lỗi hoặc quá 5 giây không được trình bày như trạng thái SAFE mới. Latest và history được xử lý riêng để mẫu lịch sử không ghi đè trạng thái hiện tại. Giao diện hiển thị nguồn, khoảng cách, severity, bằng chứng và giao dịch xác nhận.

MetaMask chỉ được gọi khi người dùng thực hiện thao tác cần ví. Giao diện kiểm tra mạng, bytecode hợp đồng và vai trò trước ghi bằng chứng; các phần tử nhận dữ liệu ngoài được escape khi render. Nút zone entry yêu cầu mẫu SAFE mới và không có yêu cầu dừng. Đây là kiểm tra ở UI; cần bổ sung ràng buộc contract/gateway nếu triển khai nhiều bên.

Gỡ khóa trên chain, gỡ trạng thái mô phỏng và trạng thái của ESP32 có ý nghĩa khác nhau. Dashboard hiện không gửi lệnh reset xuống firmware. Permit trên UI chưa được phục hồi đầy đủ sau refresh; receipt từ giao dịch ký qua trình duyệt chưa tự đồng bộ vào SQLite/outbox. Các giới hạn này được giữ rõ trong phần đánh giá.

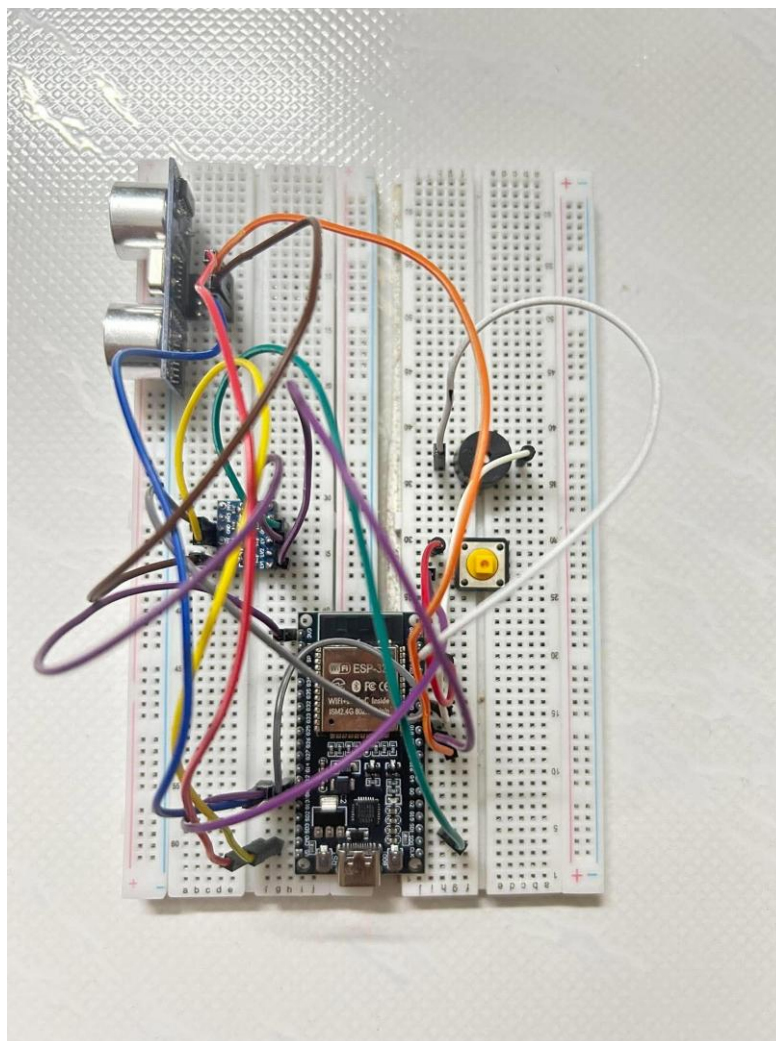
CHƯƠNG 4. KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. Môi trường và chứng cứ thực nghiệm

Kết quả AI được lấy từ output đã lưu trong notebookdedd09234 (9).ipynb chạy trên Kaggle, không thực thi lại notebook để tạo số liệu mới. Phép đánh giá sử dụng seed 42, tách bản ghi và chạy interpreter INT8 sau khi chọn model. Kết quả kiểm thử phần mềm được chạy lại ngày 05/10/2026 trên workspace project. Các nguồn này được trình bày riêng với kiểm chứng phần cứng.

Bảng 4.1. Phân loại chứng cứ

Hạng mục	Nguồn/môi trường	Giới hạn
AI float/INT8	Notebook Kaggle (9), output đã lưu	Offline; một split; chưa đo trên board
Gateway/evidence/dashboard/contracts	Bộ test Python, Node, Hardhat chạy lại	Kiểm thử phần mềm với fixture/mô phỏng
Firmware	ESP-IDF 5.5.4, esp-tflite-micro 1.4.1; artifact build_review	Build dùng model hiện có; không phải candidate mới
Lắp ráp phần cứng	Ảnh mạch trong báo cáo gốc	Có ảnh nguyên mẫu; chưa có bảng đo hiệu năng



Hình 4.1. Ảnh nguyên mẫu phần cứng được giữ từ báo cáo gốc; ảnh không thay cho phép đo độ trễ hoặc sai số.

4.2. Kết quả tầng IoT

Bảng 4.2. Trạng thái kiểm chứng tầng IoT

Hạng mục	Kết quả xác định được	Phần chưa đo
Chu kỳ dataset	Median khoảng thời gian hai mẫu ≈ 176 ms	Chưa đối chiếu jitter trên board
Chu kỳ firmware	Chờ 100 ms sau mỗi lượt đọc/xử lý	Chu kỳ thực còn có thời gian đo/suy luận
Timeout/lỗi mẫu	Code trả SENSOR_FAULT, reset history; test bảo toàn lỗi tới API đạt	Chưa có thống kê tỷ lệ timeout vật lý
Ngưỡng nguy hiểm	Code ưu tiên ≤ 30 cm và giữ EMERGENCY của AI	Chưa có thử nghiệm motor/relay
Sai số khoảng cách	Có công thức tính và ảnh mạch	Chưa có số liệu sai số theo khoảng cách/vật liệu
Độ trễ/RAM ESP32	Chưa có phép đo cho candidate	Cần đo preprocessing, Invoke, arena và toàn pipeline

Từ mã nguồn và ảnh lắp ráp có thể xác định cấu hình nguyên mẫu, nhưng chưa thể báo tỷ lệ payload hợp lệ, sai số hoặc độ trễ vật lý bằng số. Các ô chưa đo được giữ là giới hạn thực nghiệm, tránh dùng kết quả mô phỏng thay cho kết quả phần cứng.

4.3. Kết quả huấn luyện và lượng tử hóa

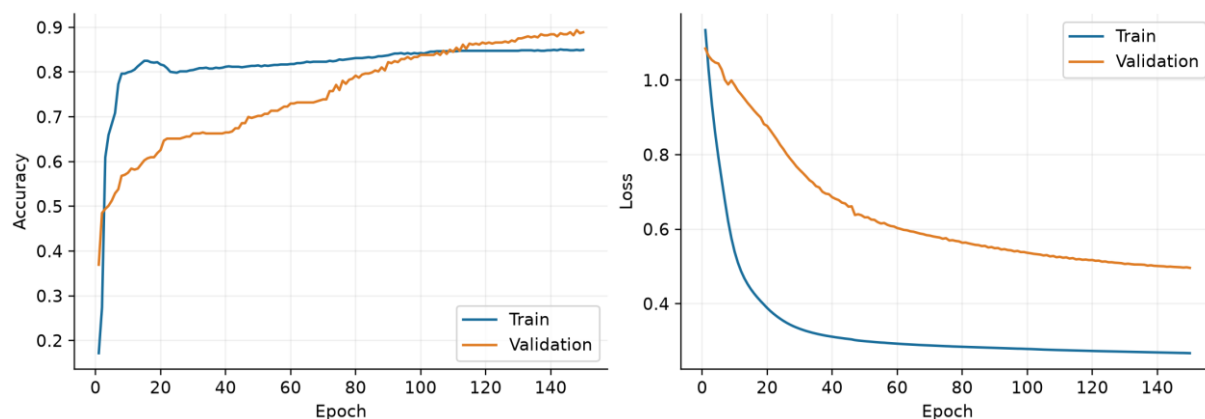
4.3.1. Phân bố dữ liệu

Bảng 4.3. Phân bố mẫu theo tập

Tập	Bản ghi	SAFE	APPROACHING	EMERGENCY	Tổng
Train	9	780	79	119	978
Validation	4	200	97	136	433
Test	4	216	88	159	463
Tổng	17	1196	264	414	1874

Mỗi source_file chỉ thuộc một tập; ba tập đều có đủ ba lớp. SAFE có 11 bản ghi, còn APPROACHING và EMERGENCY mỗi lớp chỉ có ba bản ghi. Vì vậy, mỗi tập chỉ chứa một bản ghi của từng lớp hiếm. Số mẫu nhiều hơn số phiên độc lập, và độ tin cậy của đánh giá tổng quát hóa còn bị giới hạn bởi quy mô phiên đo.

4.3.2. Diễn biến huấn luyện



Hình 4.2. Accuracy và loss của MLP 16–8, trích từ log đã lưu của notebook (9).

Ứng viên được chọn là MLP 3–16–8–3. Tại epoch 150, log ghi train accuracy 84,97%, validation accuracy 88,91%, train loss 0,2671 và validation loss 0,4961. Validation accuracy từng đạt 89,38% ở epoch 148, nhưng đó không phải số được thay cho kết quả cuối; cơ chế callback theo dõi val_loss. Train accuracy trong log được tính khi trọng số đang cập nhật và không đồng nhất với đánh giá lại toàn bộ train bằng model đã chốt.

Ở phần cuối quá trình train, validation accuracy còn tăng và validation loss tiếp tục giảm, nên chưa thấy dấu hiệu overfitting rõ qua xu hướng đã lưu. Validation accuracy cao hơn train không tự động là lỗi do thành phần và độ khó bản ghi khác nhau. Ngoài ra, training loss có class_weight còn validation loss không có trọng số lớp tương ứng; accuracy trong metrics không được tính theo class_weight [5]. Không nên kết luận overfitting chỉ từ khoảng cách giữa hai loss.

4.3.3. Kết quả test float và INT8

Bảng 4.4. Kết quả sau khi chọn mô hình

Chỉ số	Float	INT8
Accuracy	90,93%	91,14%
Macro-F1	0,8919	0,8937
Recall APPROACHING	73,86%	73,86%
Recall EMERGENCY	100%	100%
Số mẫu dự đoán đúng	421/463	422/463

Float và INT8 cho cùng nhận ở 462/463 mẫu, tương đương 99,78%. INT8 thay đổi một mẫu từ dự đoán sai sang đúng, làm accuracy tăng khoảng 0,22 điểm phần trăm. Chênh lệch này chưa chứng minh INT8 tốt hơn nói chung; kết luận phù hợp là lượng tử hóa gần như giữ nguyên chất lượng trên tập test hiện tại.

Trong gói xuất local cùng cấu hình, FlatBuffer có kích thước 3.456 byte và dùng FULLY_CONNECTED/SOFTMAX. Đây là metadata của artifact local, không phải phép đo RAM trên ESP32 hoặc xác nhận kích thước ZIP tải từ Kaggle. Khả năng chạy trên board vẫn cần kiểm tra với đúng artifact được nạp.

4.3.4. Kết quả từng lớp và ma trận nhầm lẫn

Bảng 4.5. Chỉ số từng lớp của INT8

Lớp	Precision	Recall	F1	Số mẫu
SAFE	89,59%	91,67%	0,9062	216
APPROACHING	87,84%	73,86%	0,8025	88
EMERGENCY	94,64%	100%	0,9725	159

True label	SAFE	198	9	9
	APPROACHING	23	65	0
	EMERGENCY	0	0	159
		SAFE	APPROACHING	EMERGENCY
		Predicted label		

Hình 4.3. Ma trận nhầm lẫn INT8: hàng là lớp thật, cột là lớp dự đoán.

APPROACHING là lớp yếu nhất: 65/88 mẫu đúng, còn 23 mẫu bị nhận thành SAFE, tức bỏ sót 26,14% trong tập test. Đây là hạn chế chính đối với mục tiêu cảnh báo sớm. Với SAFE, 9 mẫu bị nhận thành APPROACHING và 9 mẫu thành EMERGENCY, cho thấy vẫn có cảnh báo nhầm.

EMERGENCY đạt 159/159 mẫu đúng trong tập này. Tuy nhiên, các mẫu đều thuộc bản ghi son_ghe_30_DANGER.csv, nên kết quả không chứng minh hệ thống không bỏ sót mọi tình huống nguy hiểm. Precision EMERGENCY 94,64% cũng cho thấy recall cao không đồng nghĩa không có false alarm.

4.3.5. Mô phỏng policy và giới hạn đánh giá

Trường test_firmware_policy chạy policy bằng Python: khoảng cách ≤ 30 cm ép thành EMERGENCY, ngoài ngưỡng giữ nhẵn INT8. Kết quả bằng model INT8 trên tập hiện tại: accuracy 91,14% và recall EMERGENCY 100%. Đây là mô phỏng logic, chưa phải thí nghiệm model trên ESP32 hoặc phép đo cơ cấu dừng vật lý.

Dataset có median dt khoảng 176 ms, trong khi firmware chờ 100 ms cộng thời gian xử lý. Delta qua ba mẫu và std qua năm mẫu vì vậy có thể đại diện cho khoảng thời gian khác khi triển khai. Cần bổ sung các phiên ở đúng cadence và giữ một tập đo mới độc lập; không tiếp tục chọn cấu hình theo test đã xem.

4.4. Kết quả kiểm thử phần mềm và hợp đồng

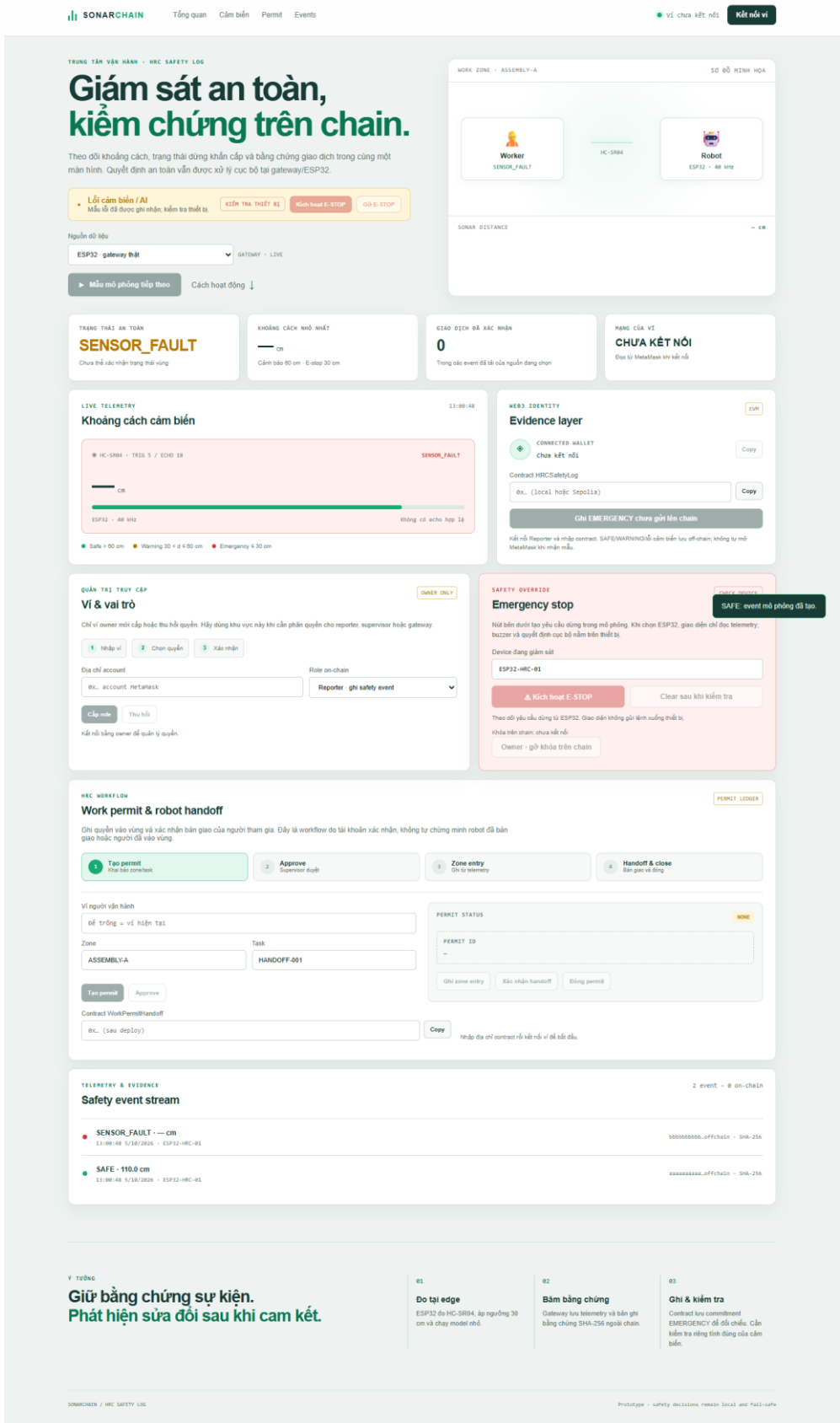
Bảng 4.6. Bộ kiểm thử chạy lại ngày 05/10/2026

Bộ kiểm thử	Kết quả	Nội dung chính
Python unittest	24/24 đạt	Evidence/hash/outbox, Serial/lỗi, SQLite/API/WebSocket, chia nhóm/feature
Node dashboard	6/6 đạt	Nguồn dữ liệu, stale/fault, reconnect, escape/hash, điều kiện zone entry
Hardhat	10/10 đạt	5 HRCSafetyLog; 3 WorkPermitHandoff; 2 SafetyLog hỗ trợ phiên bản trước

HRCSafetyLog được kiểm thử ghi sự kiện/cam kết, khóa logic, chống hash trùng, reporter trái phép, schema/thời gian/cờ dừng không hợp lệ và giữ khóa khi có SAFE tiếp theo. WorkPermitHandoff được kiểm thử vòng đời permit với handoff, chặn entry trước phê duyệt, chặn người ngoài và từ chối hash rỗng. Quyền owner của clearEmergencyStop được đối chiếu trong mã nguồn; bộ test hiện tại chưa có một ca riêng cho mọi nhánh của hàm này.

Các test đạt xác nhận hành vi trong phạm vi fixture và điều kiện đã kiểm tra. Chúng chưa thay thế việc đo tính sẵn sàng mạng, chi phí gas thực tế, đồng bộ dashboard–gateway sau giao dịch ví hoặc kiểm thử an toàn robot.

4.5. Kết quả dashboard và luồng tích hợp



Hình 4.4. Dashboard sau khi xử lý mẫu `SENSOR_FAULT` trong kiểm tra trình duyệt với API giả lập; chưa có ESP32 kết nối.

Bảng 4.7. Các tình huống được xác nhận bằng kiểm thử

Tình huống	Hành vi kiểm tra	Giới hạn chứng cứ
APPROACHING ở khoảng cách >60 cm	Gateway giữ WARNING thay vì SAFE	Fixture Serial/process_row
Edge báo SAFE nhưng khoảng cách ≤30 cm	Ngưỡng gateway giữ EMERGENCY	Kiểm thử logic, chưa đo relay
SENSOR_FAULT/null	Lỗi đi tới evidence/SQLite/API/WebSocket; UI xóa khoảng cách cũ	TestClient/DOM/API giả lập
Mẫu quá hạn	UI không hiển thị SAFE mới	Kiểm thử timestamp/stale
Nguồn mô phỏng/gateway	Tách nguồn; render không tự gửi giao dịch	Kiểm thử UI
Permit và bàn giao	Contract chuyển trạng thái theo quyền	Hardhat; chưa xác minh bàn giao vật lý

Ảnh dashboard là chứng cứ giao diện và xử lý fixture, không phải receipt blockchain hoặc phiên đo phần cứng. Báo cáo không bổ sung transaction hash, số gas hay độ trễ end-to-end chưa được thu. Giao dịch qua MetaMask vẫn cần đồng bộ trạng thái với backend; MQTT subscriber chưa lưu SQLite như đường Serial chính.

4.6. Đối chiếu mục tiêu

Bảng 4.8. Mức hoàn thành mục tiêu R1–R7

Mục tiêu	Bằng chứng	Đánh giá
R1 — IoT/buzzer	Ảnh mạch, firmware và kiểm thử xử lý lỗi	Đạt một phần; thiếu phép đo vật lý
R2 — AI/INT8	Notebook (9), interpreter test, artifact export	Đạt offline; chưa xác nhận candidate trên board
R3 — Gateway/lưu/API	Test Serial → evidence → SQLite → API/WebSocket	Đạt trong phạm vi kiểm thử phần mềm
R4 — Evidence/SHA-256	Envelope/outbox, test đối nội dung/chống trùng	Đạt trong phạm vi triển khai và test
R5 — HRCSafetyLog	5 test Hardhat cùng đối chiếu mã nguồn	Đạt trong phạm vi mạng thử cục bộ
R6 — Permit/handoff	3 test WorkPermitHandoff	Đạt workflow logic; thiếu liên kết evidence và kiểm chứng thực
R7 — Dashboard/tái lập	6 test Node, ảnh giao diện, hướng dẫn chạy	Đạt nguyên mẫu; thiếu phiên demo phần cứng đầy đủ

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề tài xây dựng nguyên mẫu SonarChain gồm tầng đo/cảnh báo, pipeline AI, gateway lưu trữ, hai hợp đồng thông minh và dashboard. Mô hình MLP 3–16–8–3 INT8 đạt accuracy 91,14% và Macro-F1 0,8937 trên tập test được tách theo bản ghi. Kết quả lượng tử hóa gần như giữ nguyên dự đoán float, cho thấy đây là ứng viên phù hợp để tiếp tục thử nghiệm trên ESP32.

Về phần mềm, 24 test Python, 6 test dashboard và 10 test Hardhat đạt trong lần kiểm tra lại. Hệ thống đã phân biệt luồng cảnh báo tại thiết bị với luồng truy vết; JSON ngoài chuỗi và cam kết SHA-256 hỗ trợ đối chiếu nội dung. Các kết quả này xác nhận nguyên mẫu và những hành vi đã kiểm thử, chưa xác nhận hệ thống an toàn chức năng công nghiệp hoặc cơ cấu dừng robot thực.

5.2. Hạn chế

Dữ liệu chỉ có 17 phiên, trong đó mỗi lớp APPROACHING/EMERGENCY có ba phiên; mỗi lớp hiếm chỉ có một phiên test. Recall APPROACHING 73,86% và 23 mẫu bị nhận thành SAFE còn hạn chế cảnh báo sớm. Recall EMERGENCY 100% thuộc một phiên test nên chưa phản ánh mọi điều kiện môi trường. Nhân theo bản ghi cũng cần được xem lại khi thu các phiên có nhiều trạng thái chuyển tiếp.

HC-SR04 chịu ảnh hưởng góc đo, bề mặt và môi trường; không cung cấp danh tính người/vật. Cadence train và firmware chưa đồng nhất, candidate chưa có kết quả RAM/latency/accuracy trên board. Buzzer có thể bị tắt âm; còi yêu cầu dừng và khóa trên chain chưa được nối với motor/relay có kiểm chứng.

Hardhat local có thể reset và không đại diện cho mạng sản xuất. Mã băm không xác nhận sự thật vật lý; khóa reporter/gateway vẫn là điểm tin cậy. Zone entry chưa ràng buộc hash với SafetyLog, một số kiểm tra thời hạn chưa áp dụng ở mọi bước. UI permit chưa phục hồi đầy đủ sau refresh, và giao dịch ví chưa tự đồng bộ backend/outbox.

5.3. Hướng phát triển

Ưu tiên thu thêm phiên APPROACHING và các tình huống gần ranh giới lớp, với nhiều tốc độ, góc đo, vật liệu và ngày đo. Thu ở cadence của firmware và giữ các phiên mới cho đánh giá độc lập.

Kiểm thử candidate trên ESP32 bằng parity vectors; đo preprocessing, Invoke, độ trễ cảnh báo, tensor arena/RAM và jitter. Chỉ thay đổi kiến trúc, số epoch hoặc class weight theo validation.

Thiết kế cơ cấu dừng motor/relay, latch và reset có kiểm soát; bổ sung watchdog và đánh giá khi mất nguồn/mạng. Nếu hướng tới ứng dụng robot thực, cần cảm biến và đánh giá rủi ro phù hợp thay cho việc suy từ ngưỡng demo.

Bổ sung mã phiên bản/hash model và thông tin thời gian vào telemetry; hoàn thiện xác thực nguồn dữ liệu, lưu bằng chứng lâu dài và công cụ đối chiếu độc lập với cam kết on-chain.

Đồng bộ receipt từ dashboard với SQLite/outbox, khôi phục permit sau refresh và liên kết telemetryEventHash với bằng chứng trong HRCSafetyLog. Bổ sung kiểm tra hết hạn cho các thao tác cần thiết.

Sau khi hoàn thành kiểm thử tích hợp, khảo sát mạng blockchain cấp phép hoặc batching/Merkle tree, đo gas và chi phí theo điều kiện triển khai; không dùng thông số Hardhat làm chi phí tiền thực.

TÀI LIỆU THAM KHẢO

- [1] E. R. Griffor, C. Greer, D. A. Wollman và M. J. Burns, Framework for Cyber-Physical Systems: Volume 1, Overview, NIST SP 1500-201, 2017, DOI 10.6028/NIST.SP.1500-201. Truy cập ngày 05/10/2026. <https://www.nist.gov/publications/framework-cyber-physical-systems-volume-1-overview>
- [2] ISO, ISO 10218-2:2025 — Robotics — Safety requirements — Part 2: Industrial robot applications and robot cells, 2025. Tham khảo phạm vi công khai. Truy cập ngày 05/10/2026. <https://www.iso.org/standard/73934.html>
- [3] Texas Instruments, Ultrasonic Sensing Basics, SLAA907D, cập nhật 12/2021. Truy cập ngày 05/10/2026. <https://www.ti.com/lit/pdf/slaa907>
- [4] Espressif Systems, ESP32 Series Datasheet, mục DC Characteristics. Truy cập ngày 05/10/2026. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [5] Keras, Dense layer; Layer activation functions; Probabilistic losses; Model training APIs. Truy cập ngày 05/10/2026. https://keras.io/api/layers/core_layers/dense/ ; <https://keras.io/api/layers/activations/> ; https://keras.io/api/losses/probabilistic_losses/ ; https://keras.io/api/models/model_training_apis/
- [6] Google AI Edge, Post-training integer quantization. Truy cập ngày 05/10/2026. https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_integer_quant
- [7] Google AI Edge, LiteRT 8-bit quantization specification. Truy cập ngày 05/10/2026. https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization_spec
- [8] scikit-learn, Cross-validation: evaluating estimator performance, phần dữ liệu có nhóm. Truy cập ngày 05/10/2026. https://scikit-learn.org/stable/modules/cross_validation.html#cross-validation-iterators-for-grouped-data
- [9] scikit-learn, Metrics and scoring: quantifying the quality of predictions, phần classification metrics. Truy cập ngày 05/10/2026. https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics
- [10] Ethereum.org, Introduction to smart contracts; Ethereum gas and fees: technical overview. Truy cập ngày 05/10/2026. <https://ethereum.org/developers/docs/smart-contracts/> ; <https://ethereum.org/developers/docs/gas/>
- [11] NIST, FIPS PUB 180-4 — Secure Hash Standard, 08/2015. Truy cập ngày 05/10/2026. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- [12] Nomic Foundation, Hardhat Network Reference, Hardhat 2, phần hardhat_reset. Truy cập ngày 05/10/2026. https://v2.hardhat.org/hardhat-network/docs/reference#hardhat_reset
- [13] HC-SR04 Ultrasonic Ranging Module, datasheet có thông tin hỗ trợ ElecFreaks, bản lưu tại SparkFun. Cần kiểm tra biến thể module thực tế. Truy cập ngày 05/10/2026. <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- [14] scikit-learn, IsolationForest API reference. Truy cập ngày 05/10/2026. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- Nguồn thực nghiệm nội bộ: notebookdded09234 (9).ipynb; pipeline và script export trong ai_model; firmware esp32_safety_idf; gateway/evidence; hai hợp đồng trong contracts/contracts; web3/app.js; các log kiểm thử ngày 05/10/2026. Báo cáo nhóm khác chỉ dùng tham khảo cách tổ chức và văn phong, không làm nguồn số liệu SonarChain.

PHỤ LỤC A. HƯỚNG DẪN TÁI LẬP

A.1. Chuẩn bị môi trường

Chạy từ thư mục gốc Blockchain. Nếu đã có .venv thì bỏ lệnh tạo lại môi trường. Hướng dẫn đầy đủ nằm trong HUONG_DAN_CHAY_PROJECT.md; các lệnh dưới đây phục vụ tái lập phần mềm và demo cục bộ.

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install -r requirements-tinyml.txt
```

A.2. Dashboard mô phỏng và blockchain local

```
.\.venv\Scripts\python.exe -m http.server 8080 --directory web3
# Mở http://localhost:8080, chọn nguồn Mô phỏng

Set-Location contracts
npm.cmd ci
npm.cmd run compile
npm.cmd test
npm.cmd run node
# Terminal khác, cùng thư mục contracts:
npm.cmd run deploy
```

Địa chỉ hợp đồng lấy từ output deploy và phải khớp mạng ví đang chọn. Khóa thử nghiệm của Hardhat chỉ dùng cho mạng phát triển; không ghi khóa thật vào báo cáo hoặc mã nguồn.

A.3. Huấn luyện và kiểm thử từ thư mục gốc

```
.\.venv\Scripts\python.exe -m ai_model.train_keras_tflite --check-data
.\.venv\Scripts\python.exe -m ai_model.train_keras_tflite --epochs 150
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
node --test tests/dashboard.test.cjs
```

Trên Kaggle, dùng ai_model/notebooks/esp32_grouped_training.ipynb và đúng dataset_3class.csv; khai báo DATASET_PATH nếu có nhiều dataset. Lưu metrics.json, history.json, split_manifest.json, model_metadata.json và ZIP export. Khi viết bảng so sánh, đọc metrics của trọng số được chốt thay vì chỉ lấy log epoch cuối.

A.4. Build và kiểm chứng candidate

Trong ESP-IDF terminal, copy đồng thời model_data.cc và model_data.h từ ai_model/artifacts/esp32_candidate/ sang esp32_safety_idf/main/, sau đó build/ nạp đúng board. Đây là bước hướng dẫn cần thực hiện khi triển khai; bản chỉnh sửa báo cáo không tự thay model hoặc nạp ESP32.

```
Set-Location esp32_safety_idf
idf.py set-target esp32
idf.py build
idf.py -p COMx flash monitor
```

Lưu artifact/hash model, cấu hình firmware, thời gian thực nghiệm, công/board, phiên đo, RAM/latency và kết quả parity. Nếu ghi chain, lưu evidence JSON, digest, network/contract và receipt tương ứng. Tính accuracy thực tế trên phiên đo mới có nhãn độc lập.

PHỤ LỤC B. MANIFEST CHIA DỮ LIỆU

Danh sách sau được lấy trực tiếp từ output notebook (9), seed 42. Đây là căn cứ để tái lập phép chia và xác định bản ghi đã được dùng trong thí nghiệm.

B.1. Train

son2_roi_50ve200_cham_RETREATING.csv
son3_roi_30ve150_cham_RETREATING.csv
son3_roi_50ve200_cham_RETREATING.csv
son3_tien_200ve100_cham_APPROACHING.csv
son_dungyen_150_SAFE.csv
son_dungyen_200_SAFE.csv
son_dungyen_20_DANGER.csv
son_trong_EMPTY.csv
son_vatnho_100_SAFE.csv

B.2. Validation

son2_roi_30ve150_cham_RETREATING.csv
son_dungyen_30_DANGER.csv
son_tien_200ve100_cham_APPROACHING.csv
son_vatnho_50_SAFE.csv

B.3. Test

son2_tien_200ve100_cham_APPROACHING.csv
son_ghe_30_DANGER.csv
son_roi_30ve150_cham_RETREATING.csv
son_roi_50ve200_cham_RETREATING.csv